What would count as a new plausible example?



Ten held-out binary MNIST images.

> What would you ask a digit generator to produce?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

What would count as a new plausible example?



Ten held-out binary MNIST images.

A new image should resemble the data without merely retrieving a training image.

> A fresh plausible digit, with variation.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Three outputs can look similar

Action	Information supplied
Copy	An existing image index
Reconstruct	An image to encode
Sample	Randomness and a fitted model

> What information does each procedure need?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Three outputs can look similar

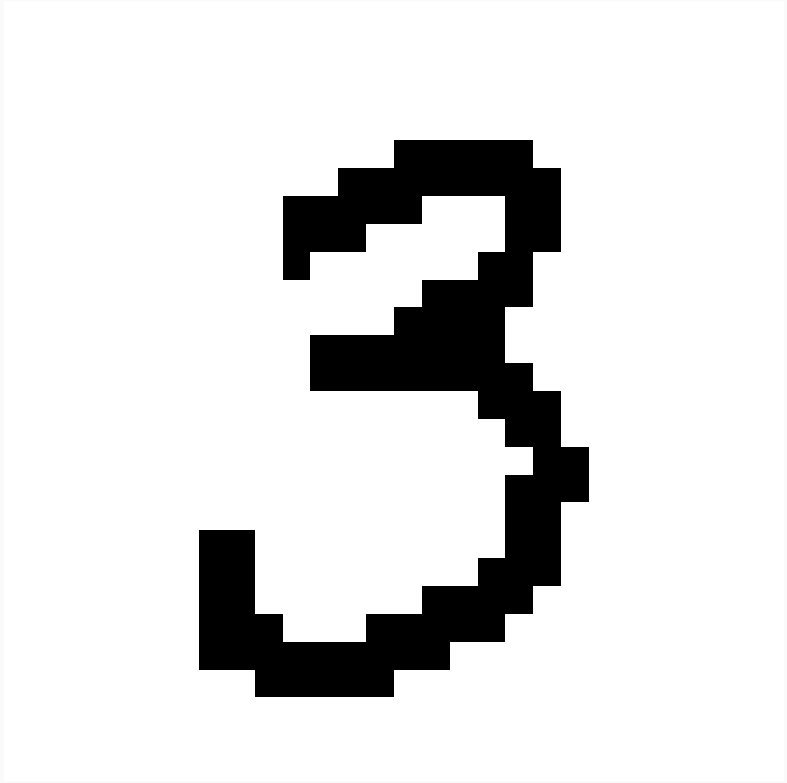
Action	Information supplied
Copy	An existing image index
Reconstruct	An image to encode
Sample	Randomness and a fitted model

The input to the procedure tells us which task it performs.

> Sampling does not require an input image.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Keep this handwritten 3 in view



Observed x

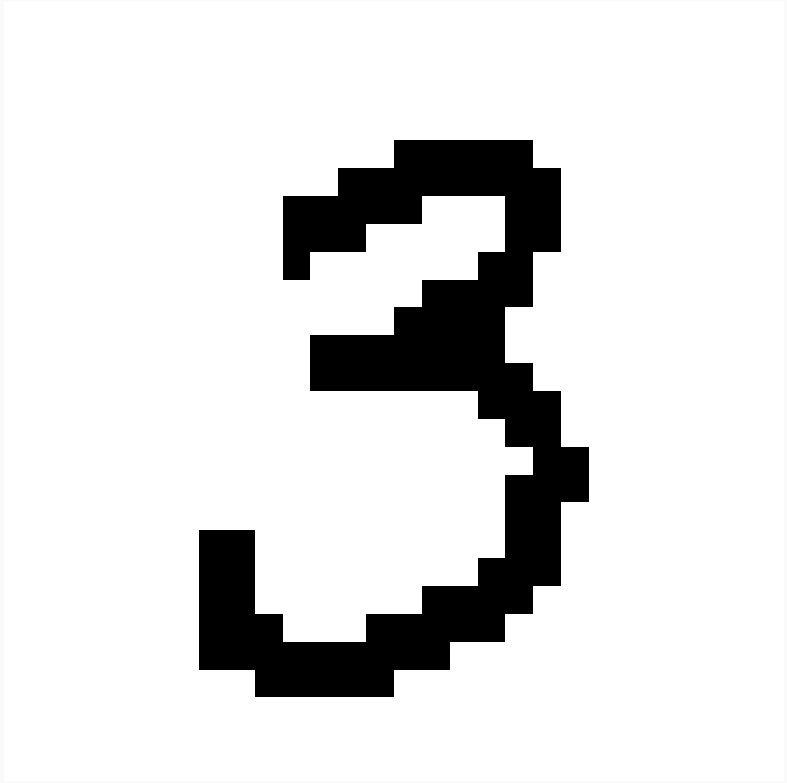


A model output

> Is the right image evidence that we can sample a new digit?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Keep this handwritten 3 in view



Observed x



A model output

The right image was computed from the left. It is a reconstruction.

> It shows reconstruction; a sampling procedure is still missing.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

We see examples, not the entire distribution

$$x_1, \ldots, x_N \sim p_{\text{data}}(x)$$

> What must the fitted model support?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

We see examples, not the entire distribution

$$x_1, \ldots, x_N \sim p_{\text{data}}(x)$$

$$\text{fit } p_\theta \quad \longrightarrow \quad x_{\text{new}} \sim p_\theta$$

A rule for producing new examples.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One convincing example is not enough

A generator always returns the same beautiful 3.

> Would you call that a successful digit generator?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One convincing example is not enough

A generator always returns the same beautiful 3.

Test	Question
Plausibility	Does each output look like a digit?
Coverage	Which kinds of digits and handwriting occur?

> It may produce a plausible image but misses almost all variation.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Fit a distribution. Then challenge its samples.



> Which part of this route is already familiar from Lecture 1?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Fit a distribution. Then challenge its samples.

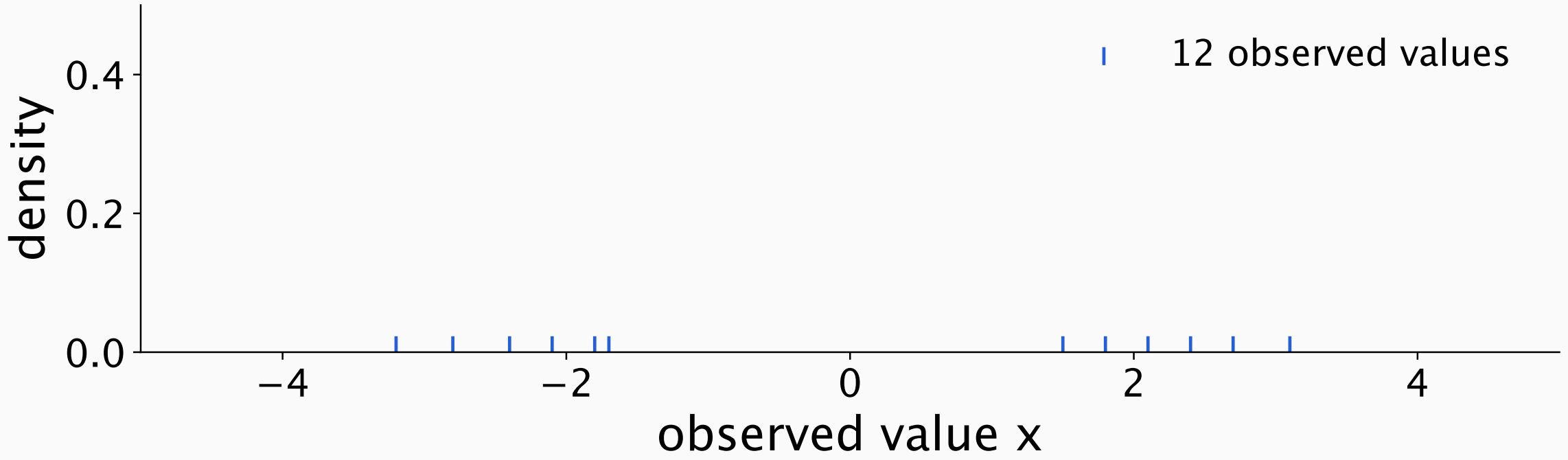


Our final test will be: where do the decoder's input codes come from?

> Choosing a distribution family and fitting its parameters.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

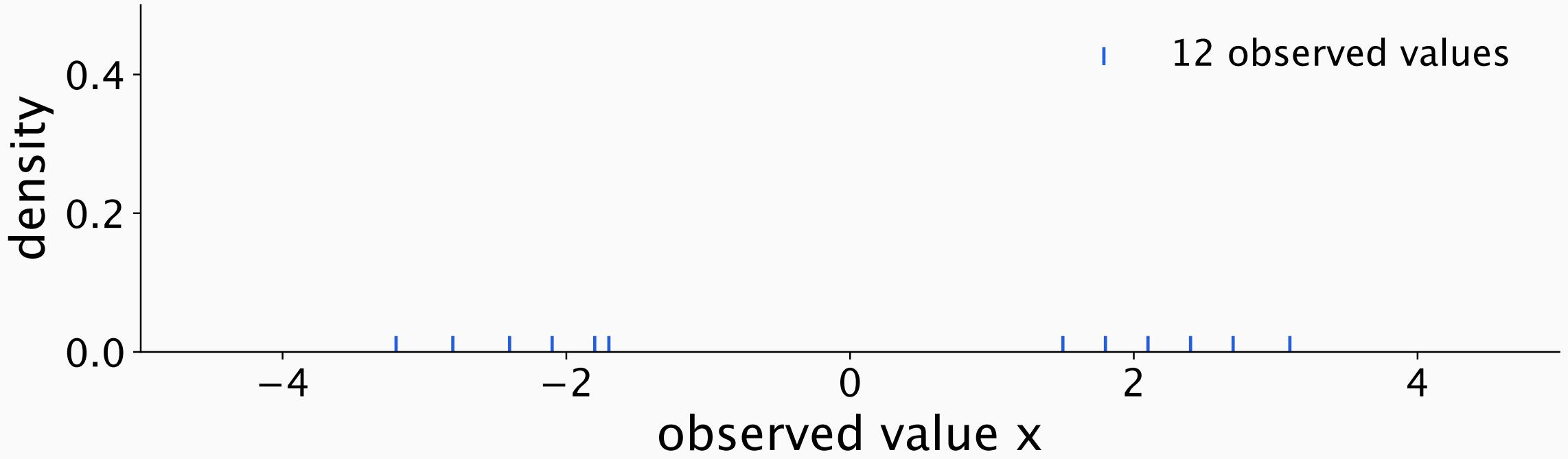
Twelve observations suggest two groups



Where should a new sample usually land?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Twelve observations suggest two groups



A useful model should put probability near both groups.

> Near either observed group, rather than mainly between them.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Fit the mean and the spread

$$\hat{\mu} = \frac{1}{N}\sum_i x_i, \qquad \hat{\sigma}^2 = \frac{1}{N}\sum_i (x_i - \hat{\mu})^2$$

> Is the fitted centre near either group?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Fit the mean and the spread

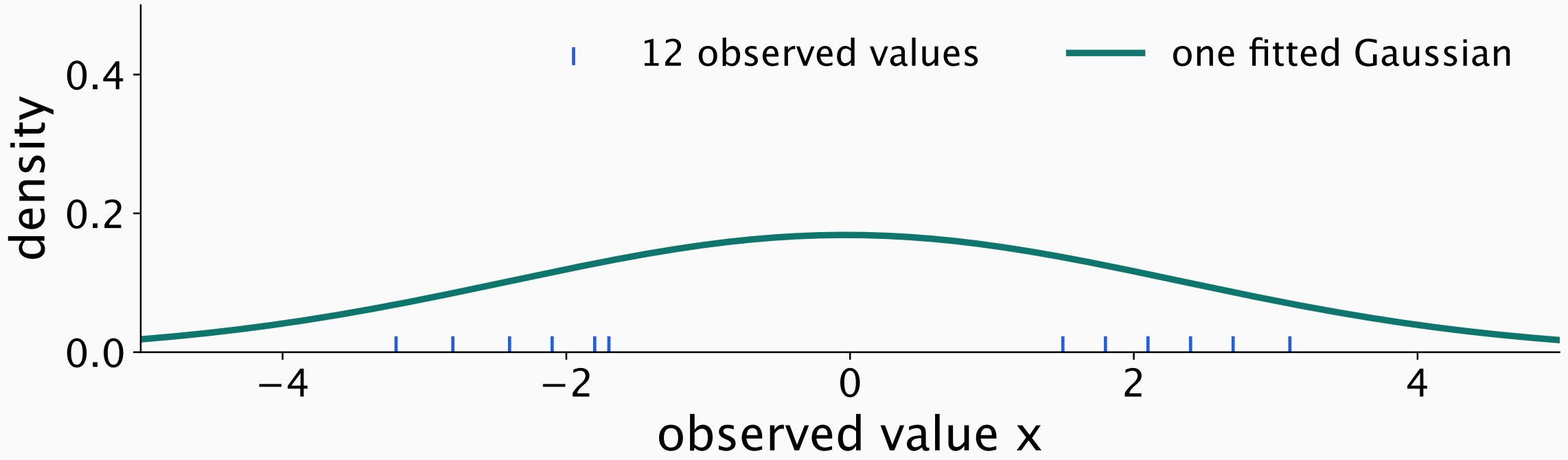
$$\hat{\mu} = \frac{1}{N}\sum_i x_i, \qquad \hat{\sigma}^2 = \frac{1}{N}\sum_i (x_i - \hat{\mu})^2$$

$$\hat{\mu} \approx -0.033, \qquad \hat{\sigma} \approx 2.362$$

> It lies between the groups; a single Gaussian pools both.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

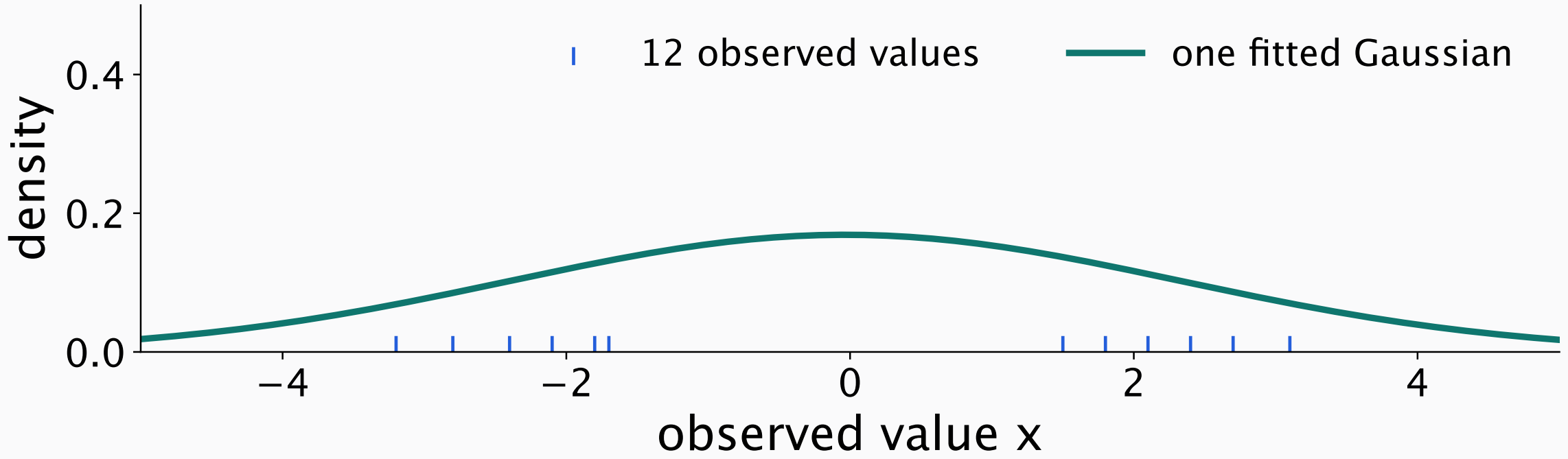
The fitted centre is in the gap



> What structure did this family discard?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The fitted centre is in the gap



A good summary of mean and variance need not capture the shape.

> The two distinct groups.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Sampling is a separate computation

```
epsilon = rng.normal()
x_new = mean + std * epsilon
```

Which values stay fixed while we generate many new x?

Sampling is a separate computation

```
epsilon = rng.normal()
x_new = mean + std * epsilon
```

$$\epsilon \sim \mathcal{N}(0, 1), \qquad x_{\text{new}} = \hat{\mu} + \hat{\sigma}\epsilon$$

The fitted mean and scale; only epsilon changes.

First choose a group, then a value



What extra random choice creates two groups?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

First choose a group, then a value



$$c \sim \text{Categorical}(\pi), \quad x \mid c \sim \mathcal{N}(\mu_c, \sigma_c^2)$$

The component choice c.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The hidden choice becomes a sum

$$p(x) = \sum_{c=1}^{2} \pi_c \, p(x \mid c)$$

> Why do we add the two possibilities?

Computed 1D two-component EM fit, fixed 12 observations, 40 updates; scripts/build_evidence.py.

The hidden choice becomes a sum

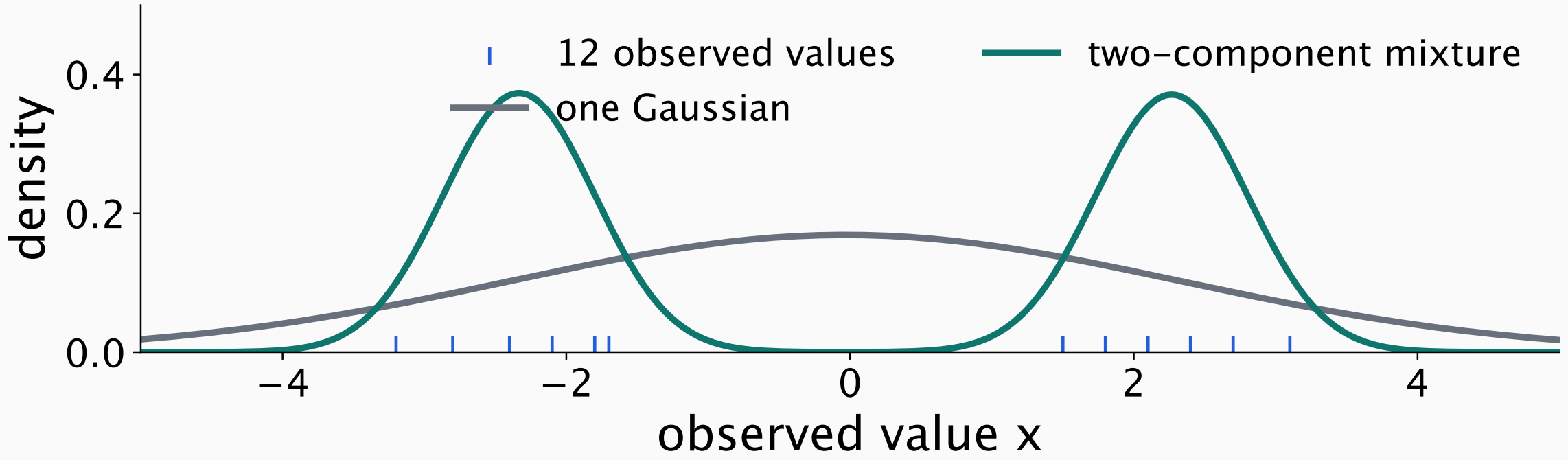
$$p(x) = \sum_{c=1}^{2} \pi_c \, p(x \mid c)$$

Fitted quantity	Component 1	Component 2
Weight	0.500	0.500
Mean	-2.333	2.267
Std	0.534	0.537

> The component is unobserved; marginalisation sums its alternatives.

Computed 1D two-component EM fit, fixed 12 observations, 40 updates; scripts/build_evidence.py.

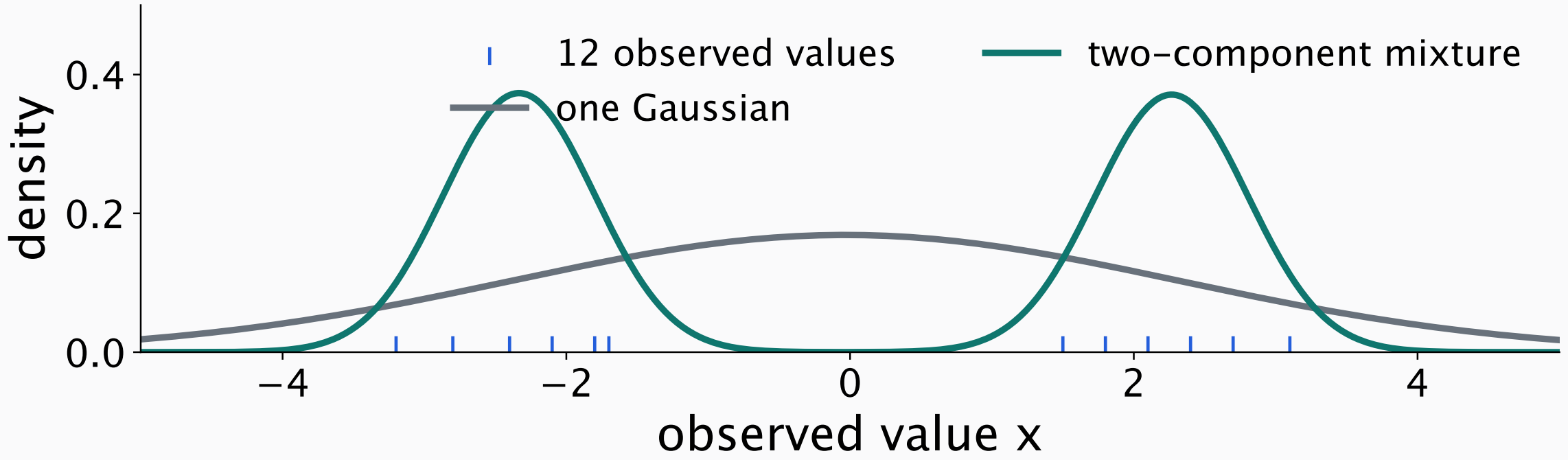
A richer family can express both groups



Where has the density changed most?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A richer family can express both groups

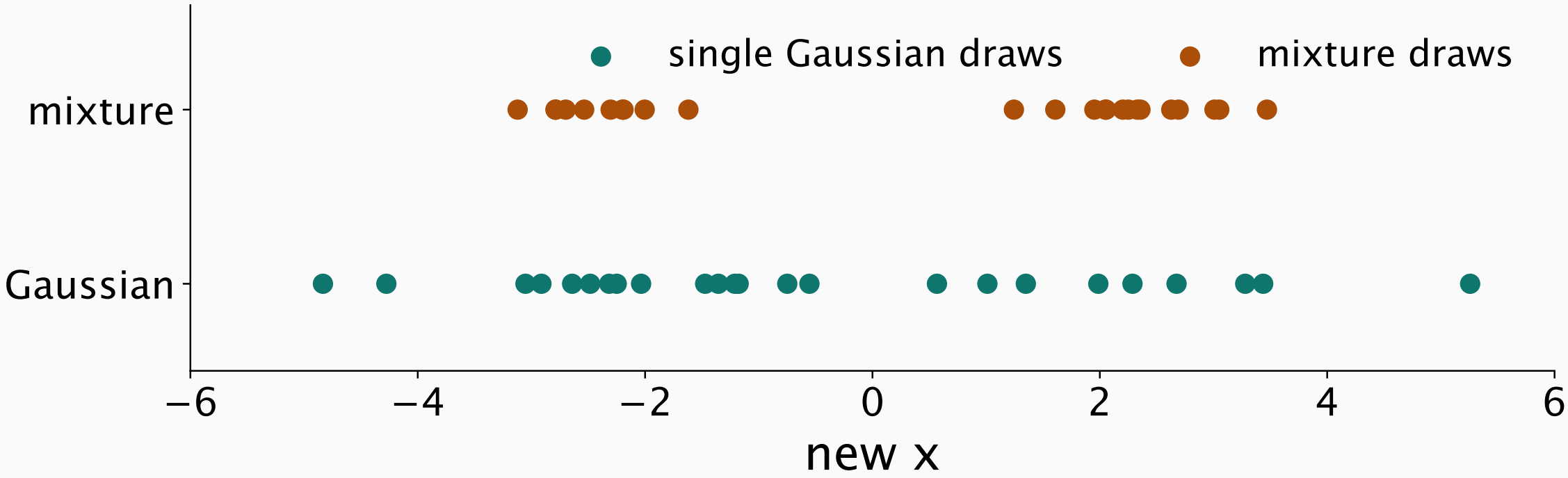


The model family changed; the observations did not.

> The mixture concentrates around the two groups.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

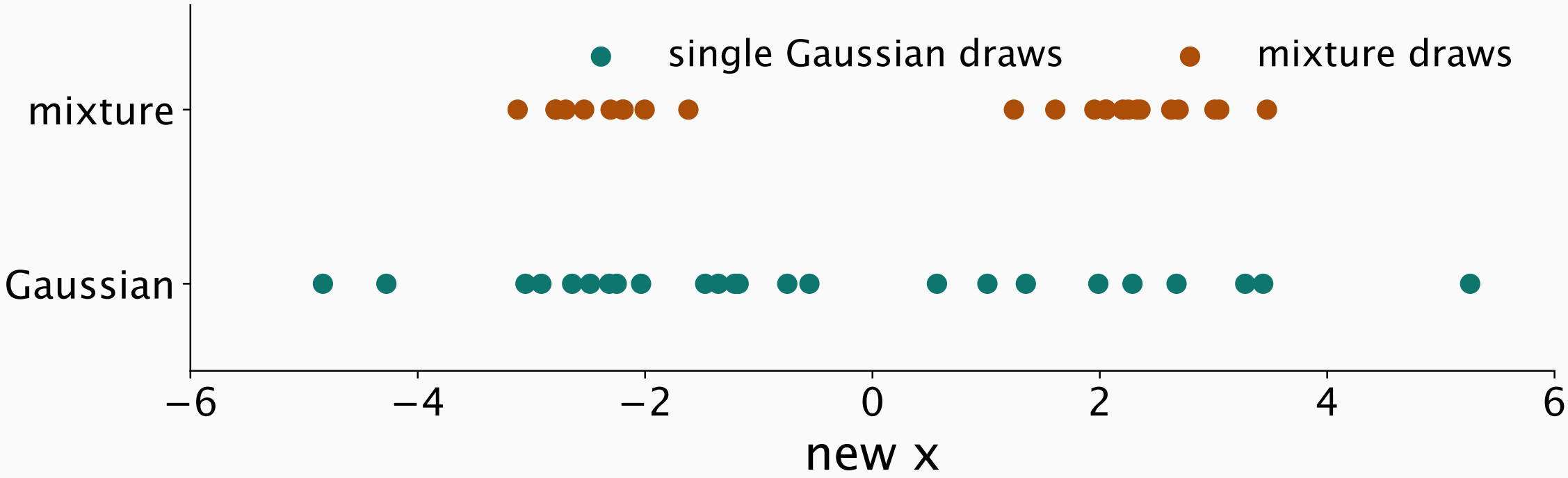
Look at the samples the models produce



Which row better reflects the two groups?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Look at the samples the models produce

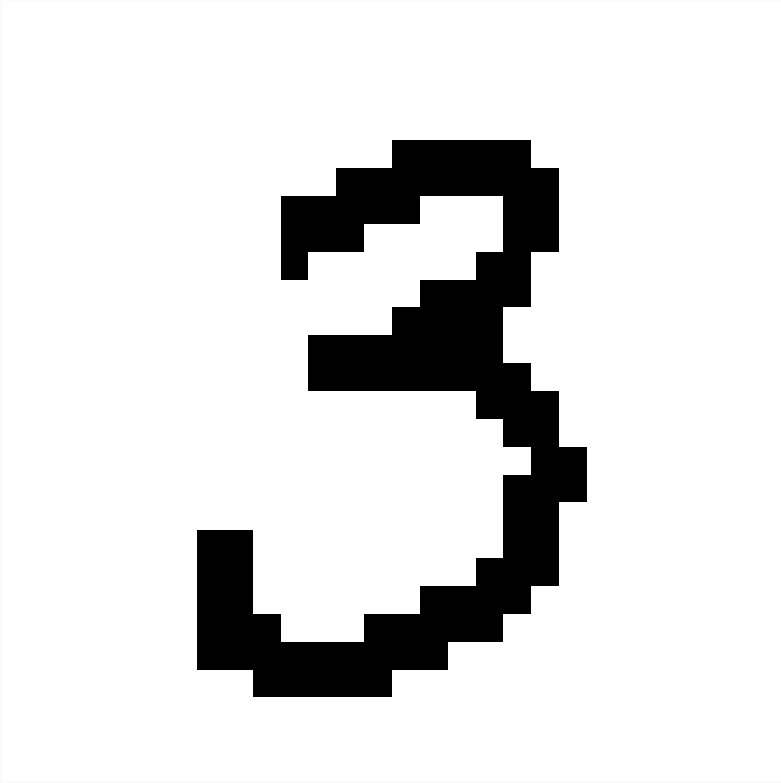


A finite sample is evidence about this run, not a complete test of the distribution.

> The mixture row follows the two-mode structure.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

An image is 784 observed values



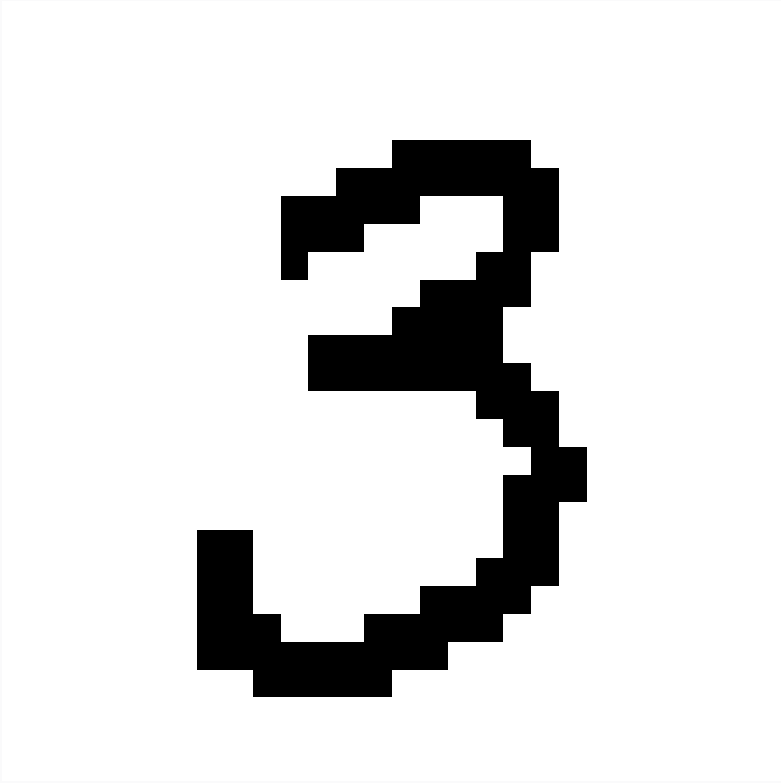
Official MNIST test index 90.

$$x \in \{0,1\}^{28\times 28} \quad \leftrightarrow \quad x \in \{0,1\}^{784}$$

> What information was removed by thresholding?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

An image is 784 observed values



$$x \in \{0,1\}^{28\times 28} \quad \leftrightarrow \quad x \in \{0,1\}^{784}$$

Official MNIST test index 90.

We threshold grey levels at 0.5 once. These are fixed binary observations.

> Grey intensity levels; the spatial arrangement remains.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One pixel has a simple distribution

$$x_j \sim \text{Bernoulli}(\pi_j)$$

> If πj is 0.8, what does the model say?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One pixel has a simple distribution

$$x_j \sim \text{Bernoulli}(\pi_j)$$

$$p(x_j) = \pi_j^{x_j}(1-\pi_j)^{1-x_j}$$

> The pixel is one with probability 0.8, not always grey 0.8.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Fit one probability per pixel



Mean of 256 training images.

$$\hat{\pi}_j = \frac{1}{N} \sum_i x_{ij}$$

> Is this mean image itself a typical observed digit?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Fit one probability per pixel



$$\hat{\pi}_j = \frac{1}{N} \sum_i x_{ij}$$

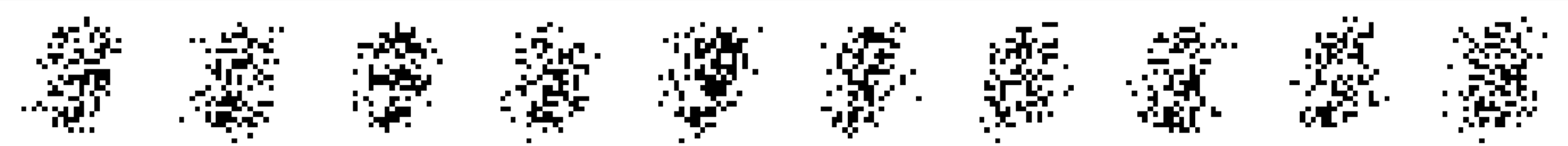
Mean of 256 training images.

This probability image averages over many different digit identities.

> No; averaging can combine incompatible strokes.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Correct pixel frequencies can make poor images

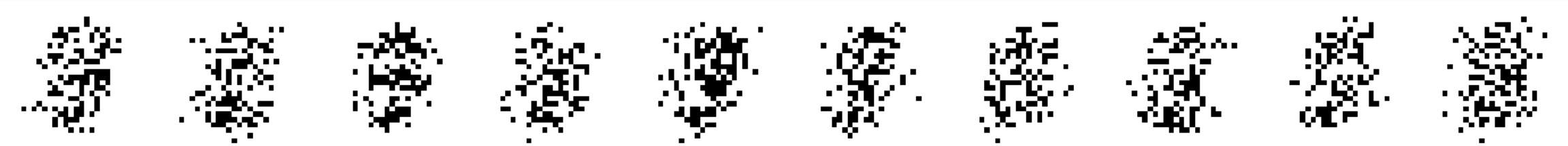


Computed Bernoulli draws from fitted marginal pixel frequencies.

> What relation between pixels is missing?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Correct pixel frequencies can make poor images



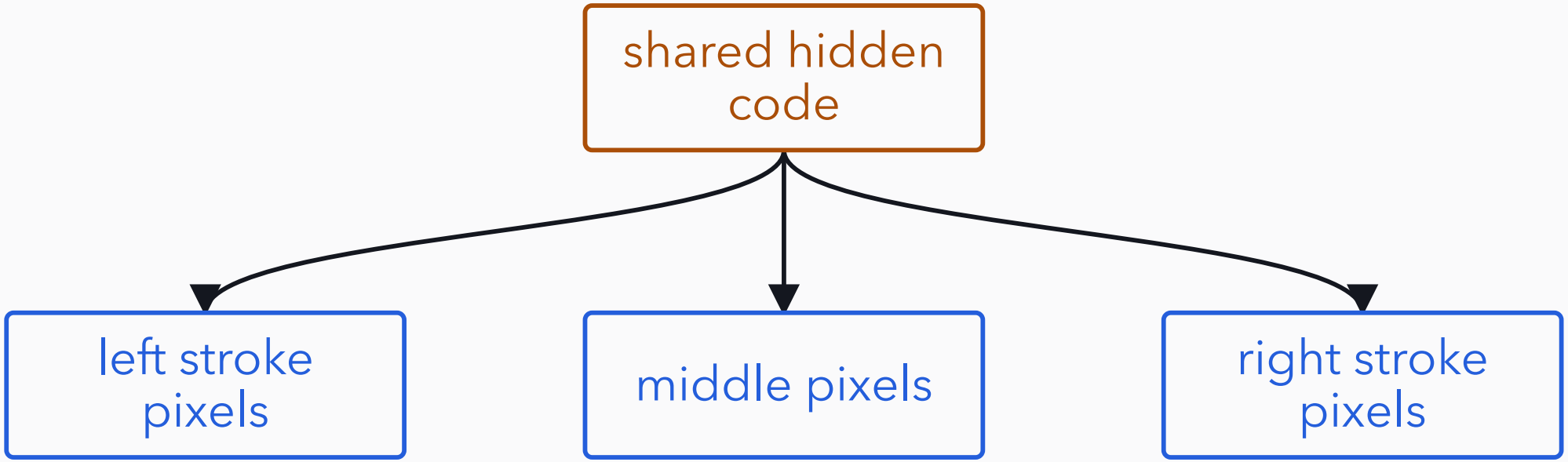
Computed Bernoulli draws from fitted marginal pixel frequencies.

$$p(x) = \prod_{j=1}^{784} \text{Bernoulli}(x_j; \hat{\pi}_j)$$

> Whether neighbouring strokes and distant parts belong to the same digit.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

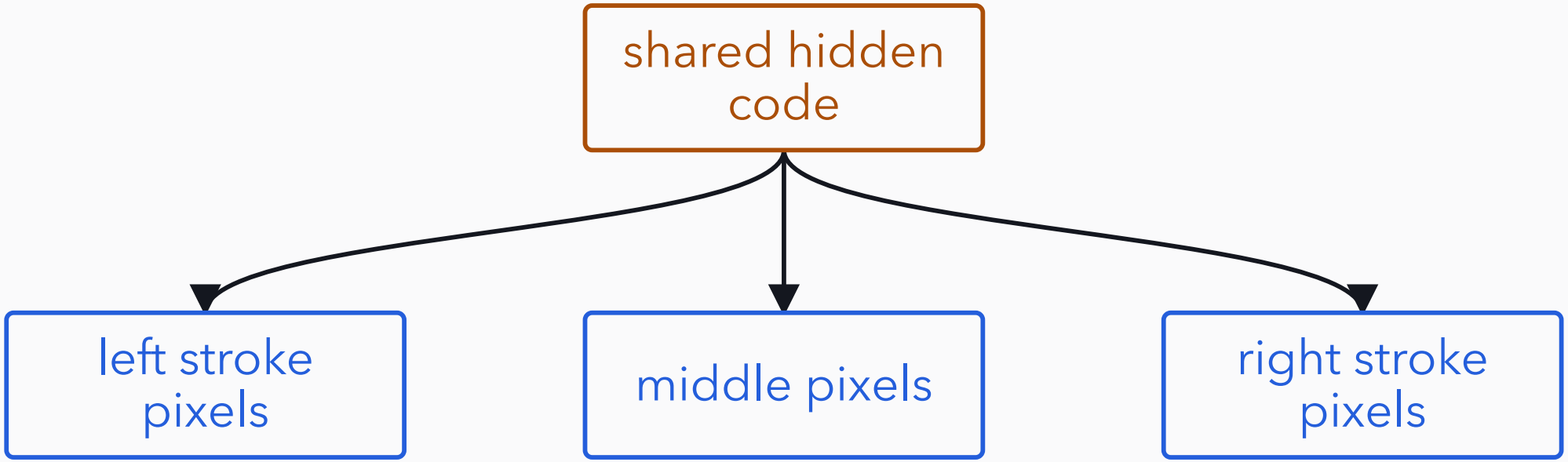
Strokes are coordinated choices



What was the analogous hidden choice in the mixture?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Strokes are coordinated choices



A shared latent choice can coordinate many output pixels.

> The component index c controlled where x was likely to fall.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A low-dimensional code can reconstruct a vector

$$z = W_{\text{enc}} x, \qquad \hat{x} = W_{\text{dec}} z$$

> What restriction does a linear decoder impose?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A low-dimensional code can reconstruct a vector

$$z = W_{\text{enc}} x, \qquad \hat{x} = W_{\text{dec}} z$$

$$784 \longrightarrow 2 \longrightarrow 784$$

> Its reconstructions lie in a low-dimensional linear subspace.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

PCA is a useful linear reference

Centred data · linear encoder and decoder · squared reconstruction error

> Would every neural autoencoder automatically equal PCA?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

PCA is a useful linear reference

Centred data · linear encoder and decoder · squared reconstruction error

Under the usual rank assumptions, a global optimum spans the principal subspace.
Nonlinear networks can learn a curved representation.

> No. Linearity, squared error and optimisation assumptions matter.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Let a network compute the code



> Is z a stored parameter for this particular image?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Let a network compute the code



$$z = E_\phi(x)$$

> It is a computed activation; φ are the shared stored weights.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A second network maps the code back



> Why are there 784 outputs again?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A second network maps the code back



$$a = G_\theta(z), \qquad \hat{x} = \mathrm{sigmoid}(a)$$

Each output corresponds to a pixel in the reconstruction.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Two kinds of quantities change

Quantity	Role
x	Observed input and target
φ, θ	Stored learned weights
z	Computed code for this x
a, $\hat{x}$	Computed logits and reconstruction

> Which quantities does the optimiser store and update?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Two kinds of quantities change

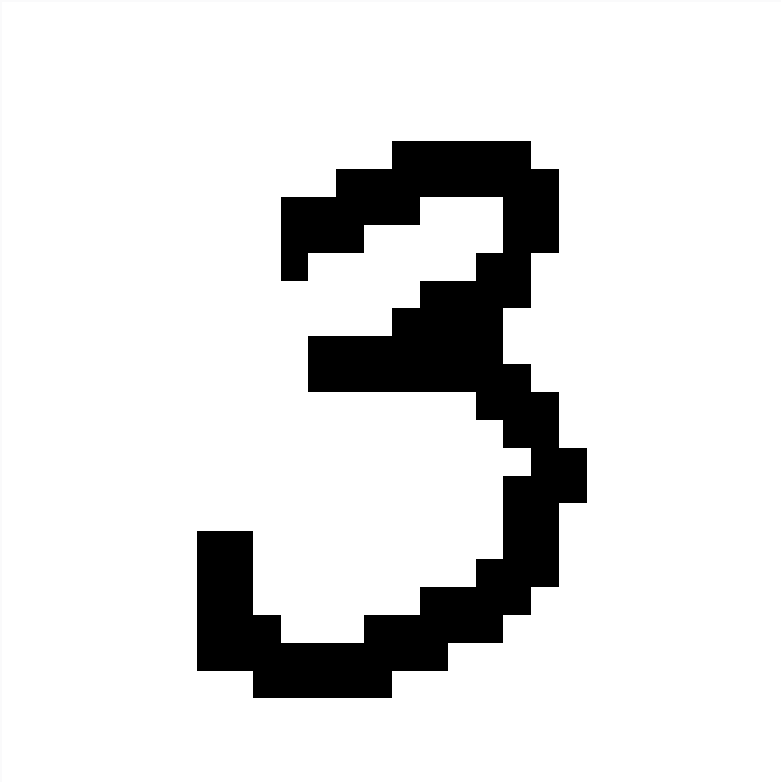
Quantity	Role
x	Observed input and target
φ, θ	Stored learned weights
z	Computed code for this x
a, $\hat{x}$	Computed logits and reconstruction

A new image changes the activations even when the weights stay fixed.

> The encoder and decoder weights.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Our 3 becomes two numbers

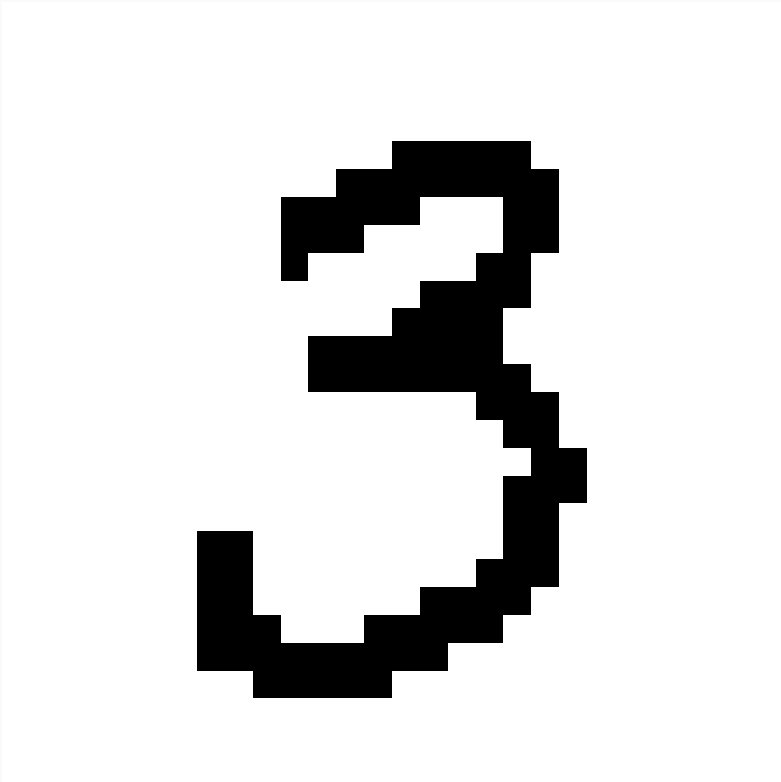


$$E_\phi(x) \approx \begin{bmatrix} 0.293 \\ 1.216 \end{bmatrix}$$

> Do the coordinates have to mean digit identity and slant?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Our 3 becomes two numbers



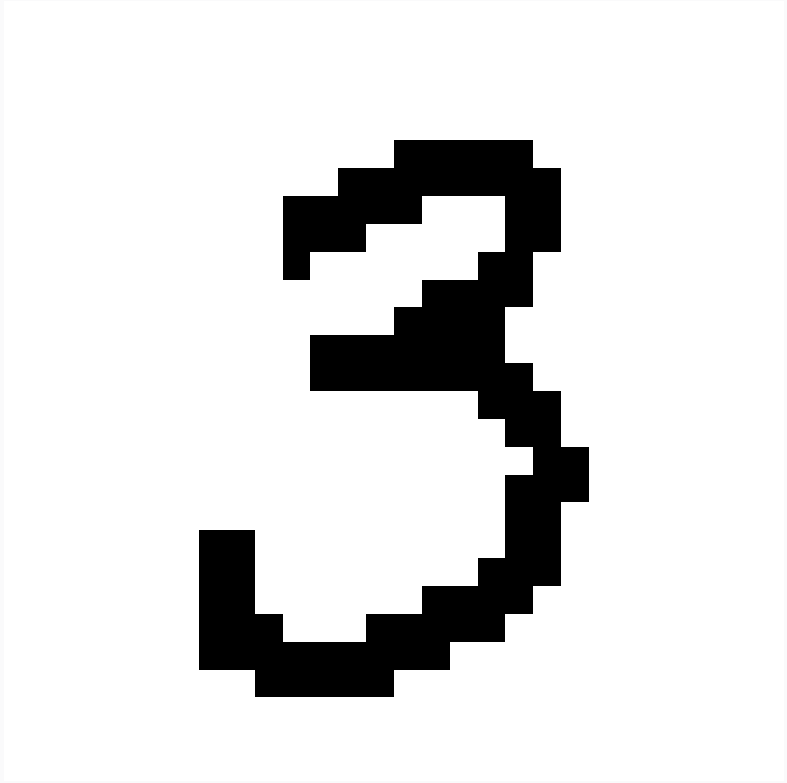
$$E_\phi(x) \approx \begin{bmatrix} 0.293 \\ 1.216 \end{bmatrix}$$

These are measured coordinates from the trained two-dimensional AE.

> No. Coordinate meanings are learned and need not align with named factors.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Decode the code that the encoder produced



Observed binary image

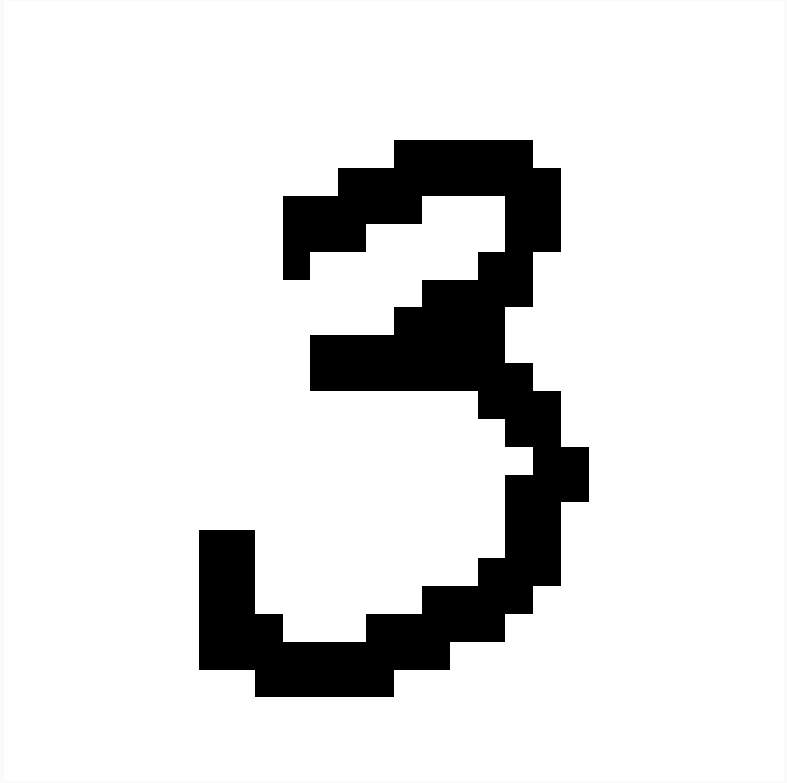


Decoder probabilities at Eφ(x)

> What should the training target be?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Decode the code that the encoder produced



Observed binary image



Decoder probabilities at Eφ(x)

This succeeds only if both learned transformations work together.

> The same original image x.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The input supplies its own target

$$x \xrightarrow{E_\phi} z \xrightarrow{G_\theta} a \quad \text{compared with } x$$

> Could this training rule work if digit labels were unavailable?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The input supplies its own target

$$x \xrightarrow{E_\phi} z \xrightarrow{G_\theta} a \quad \text{compared with } x$$

Digit labels are not used by this reconstruction objective.

> Yes. The observed image itself supplies the target.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Two values must carry enough information

$$784 \to 512 \to 256 \to 2 \to 256 \to 512 \to 784$$

> What do you expect to lose first with a small code?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Two values must carry enough information

$$784 \to 512 \to 256 \to 2 \to 256 \to 512 \to 784$$

A small code limits this architecture, but it is not a theorem that useful factors will be discovered.

> Fine details may be lost while broad shape remains.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Predict before looking at reconstructions

Model	Same task
AE with 2 code values	Reconstruct unseen binary digits
AE with 16 code values	Reconstruct the same unseen digits

> Which should have more capacity to preserve handwriting detail?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Predict before looking at reconstructions

Model	Same task
AE with 2 code values	Reconstruct unseen binary digits
AE with 16 code values	Reconstruct the same unseen digits

Keep input representation and per-image loss identical.

> The 16-dimensional code has more capacity, though training still matters.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Start with one observed pixel

$$x_j = 1, \qquad \hat{x}_j = 0.8$$

> What probability should increase to reduce this loss?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Start with one observed pixel

$$x_j = 1, \qquad \hat{x}_j = 0.8$$

$$-\log p(x_j \mid z) = -\log(0.8) \approx 0.223$$

> The probability of the observed one.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A zero uses the other probability

$$x_j = 0, \qquad \hat{x}_j = 0.1$$

Why is the loss not $-\log(0.1)$?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A zero uses the other probability

$$x_j = 0, \qquad \hat{x}_j = 0.1$$

$$-\log p(x_j \mid z) = -\log(0.9) \approx 0.105$$

> The observation is zero, whose probability is 1−0.1.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One expression covers both outcomes

$$\ell_j = -x_j \log \hat{x}_j - (1 - x_j) \log(1 - \hat{x}_j)$$

> Which term survives for an observed one?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One expression covers both outcomes

$$\ell_j = -x_j \log \hat{x}_j - (1 - x_j) \log(1 - \hat{x}_j)$$

The observed bit selects the appropriate term. Logits-based code evaluates this stably.

> Only −log x̂j.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Compute a four-pixel reconstruction loss

Pixel	x	$\hat{x}$	Contribution
1	1	0.8	0.223
2	0	0.1	0.105
3	1	0.6	0.511
4	0	0.3	0.357

> Do we add or average these four event contributions?

Chosen four-pixel example. Exact BCE independently checked against the Bernoulli log-likelihood in notebooks/lab.py.

Compute a four-pixel reconstruction loss

Pixel	x	$\hat{x}$	Contribution
1	1	0.8	0.223
2	0	0.1	0.105
3	1	0.6	0.511
4	0	0.3	0.357

$$\ell(x, \hat{x}) \approx 1.196 \text{ nats}$$

> Here we add them; they belong to one image.

Chosen four-pixel example. Exact BCE independently checked against the Bernoulli log-likelihood in notebooks/lab.py.

Sum within an image; average over images

$$\ell_i = \sum_{j=1}^{784} \ell_{ij}$$

What unit does the final number have?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Sum within an image; average over images

$$\ell_i = \sum_{j=1}^{784} \ell_{ij}$$

$$L = \frac{1}{B} \sum_{i=1}^{B} \ell_i$$

Mean reconstruction negative log-likelihood in nats per image.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The code has the same two reductions

```
logits = decoder(encoder(x))
pixel_loss = BCEWithLogits(logits, x,
                           reduction="none")
```

Which axis is summed and which is averaged?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The code has the same two reductions

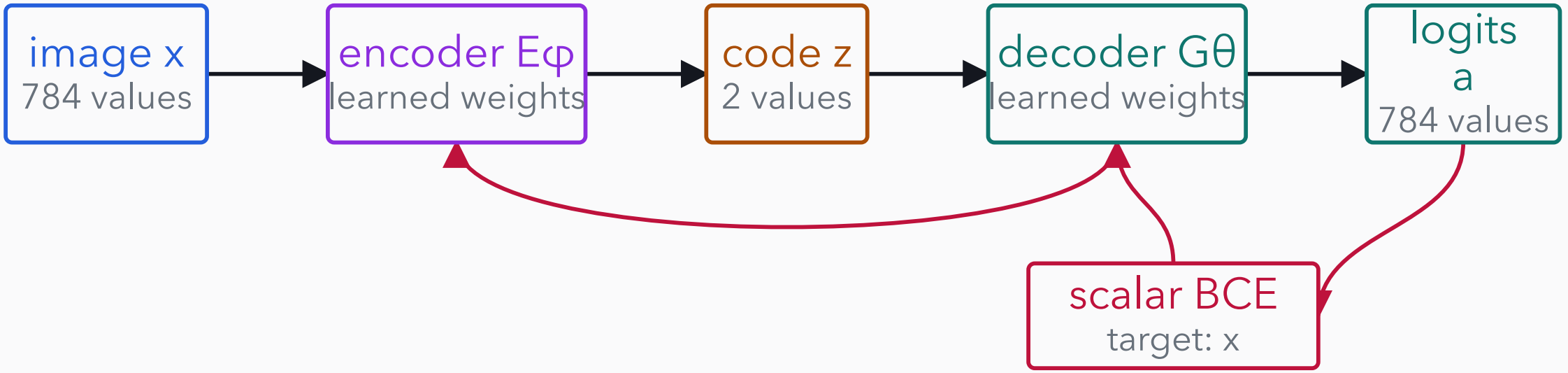
```
logits = decoder(encoder(x))
pixel_loss = BCEWithLogits(logits, x,
                           reduction="none")
```

```
per_image = pixel_loss.sum(dim=1)
loss = per_image.mean()
```

Sum 784 pixels for each image; average the batch.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

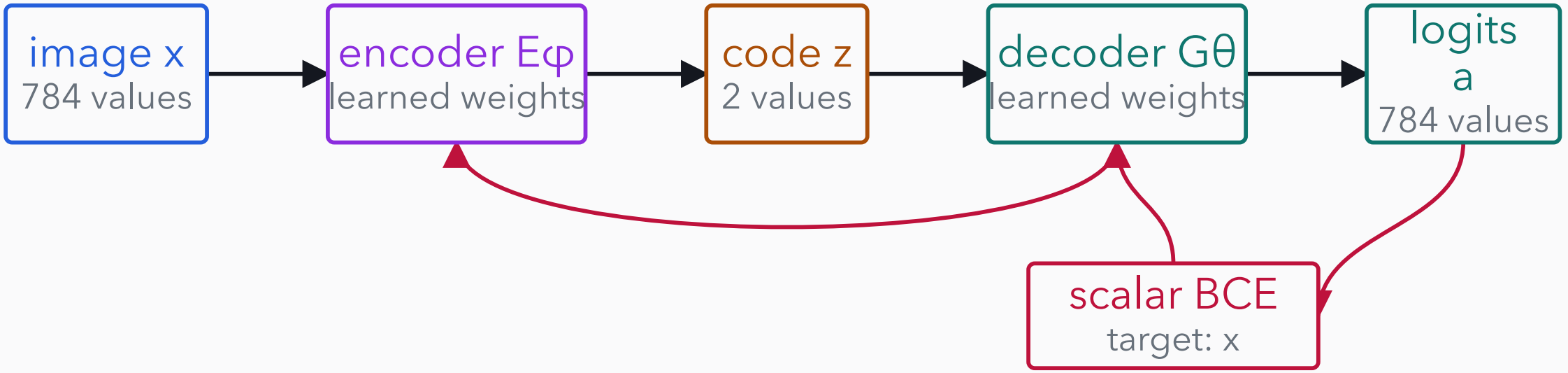
Both networks receive the reconstruction signal



Does the encoder need a separate target code?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Both networks receive the reconstruction signal



The decoder gradient travels through z into the encoder.

> No. The reconstruction loss supplies a gradient through the decoder.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

One actual SGD step changes both networks

```
loss.backward()
with torch.no_grad():
    for parameter in model.parameters():
        parameter -= eta * parameter.grad
```

What evidence confirms that the update happened?

Actual float64 SGD replay on copied ae2 checkpoint. data/fixtures.json; qa/science-checks.json.

One actual SGD step changes both networks

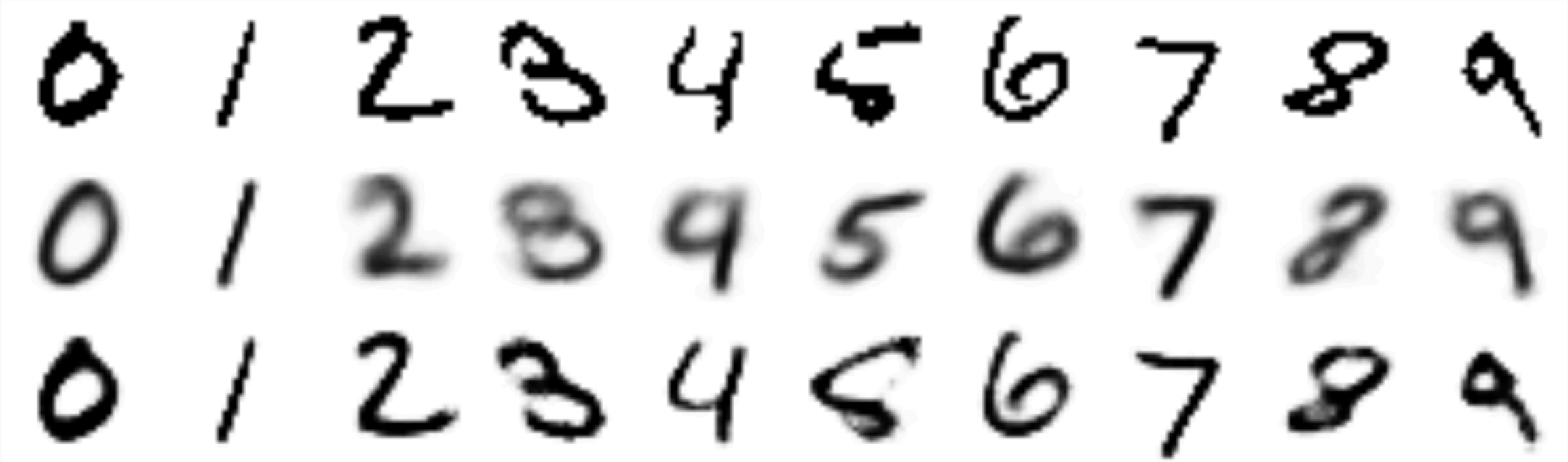
```
loss.backward()
with torch.no_grad():
    for parameter in model.parameters():
        parameter -= eta * parameter.grad
```

Setting	Value
Training batch	8 images, official train split
Step size	$\eta = 0.0001$
Before → after	121.438 → 121.087 nats/image

The parameter-delta checks and recomputed batch loss.

Actual float64 SGD replay on copied ae2 checkpoint. data/fixtures.json; qa/science-checks.json.

Test the prediction on unseen images

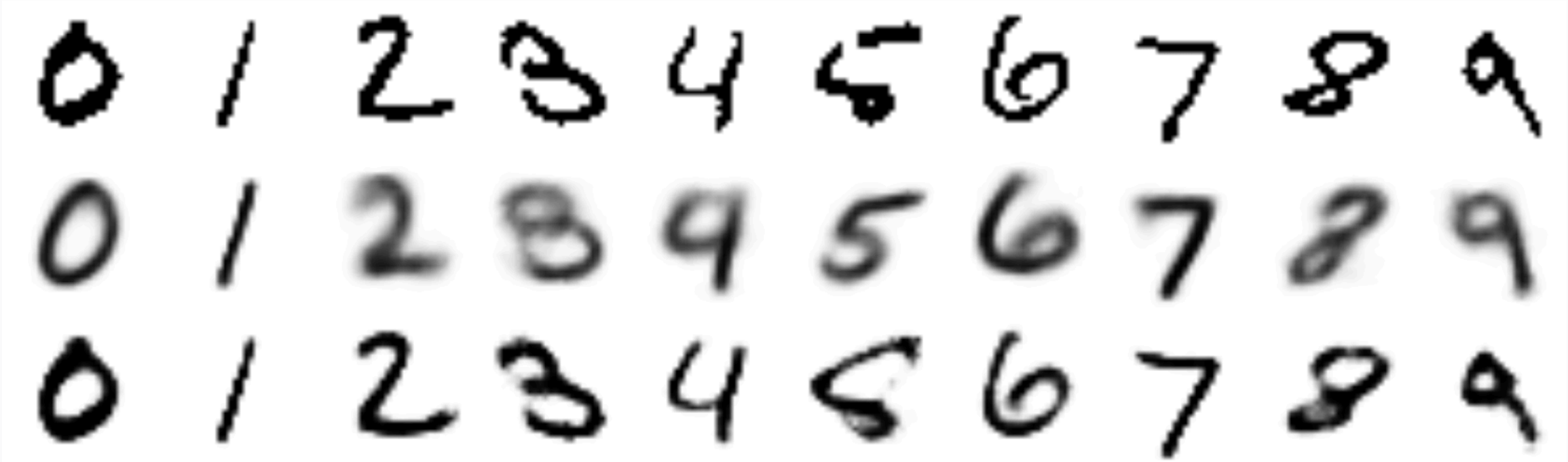


Top: observed. Middle: AE d=2. Bottom: AE d=16. Same ten test images.

> Where does the larger code preserve visible detail?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Test the prediction on unseen images



Top: observed. Middle: AE d=2. Bottom: AE d=16. Same ten test images.

Inspect detail as well as the scalar loss. These are held-out reconstructions.

> Identify concrete stroke differences in these examples.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Measure reconstruction on the same test subset

Model	Mean pixel-summed BCE
AE d=2	127.617
AE d=16	44.625

> Does lower reconstruction loss prove better prior samples?

Fresh CPU replay of existing 30-epoch AE checkpoints; test0:2000, no test updates.

Measure reconstruction on the same test subset

Model	Mean pixel-summed BCE
AE d=2	127.617
AE d=16	44.625

Recorded checkpoints · binary test images 0–1999 · nats per image.

> No. This evaluates reconstruction at encoded inputs.

Fresh CPU replay of existing 30-epoch AE checkpoints; test0:2000, no test updates.

The decoder can accept any two numbers



> Where should a new code come from?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

The decoder can accept any two numbers



$$z_{\text{new}} \overset{?}{\sim} \mathcal{N}(0, I)$$

> The reconstruction objective has not specified a distribution for it.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Try standard-normal codes anyway

```
z_new = torch.randn(10, 2)
logits = decoder(z_new)
probability_images = logits.sigmoid()
```

Should these look as good as reconstructions?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Try standard-normal codes anyway

```
z_new = torch.randn(10, 2)
logits = decoder(z_new)
probability_images = logits.sigmoid()
```

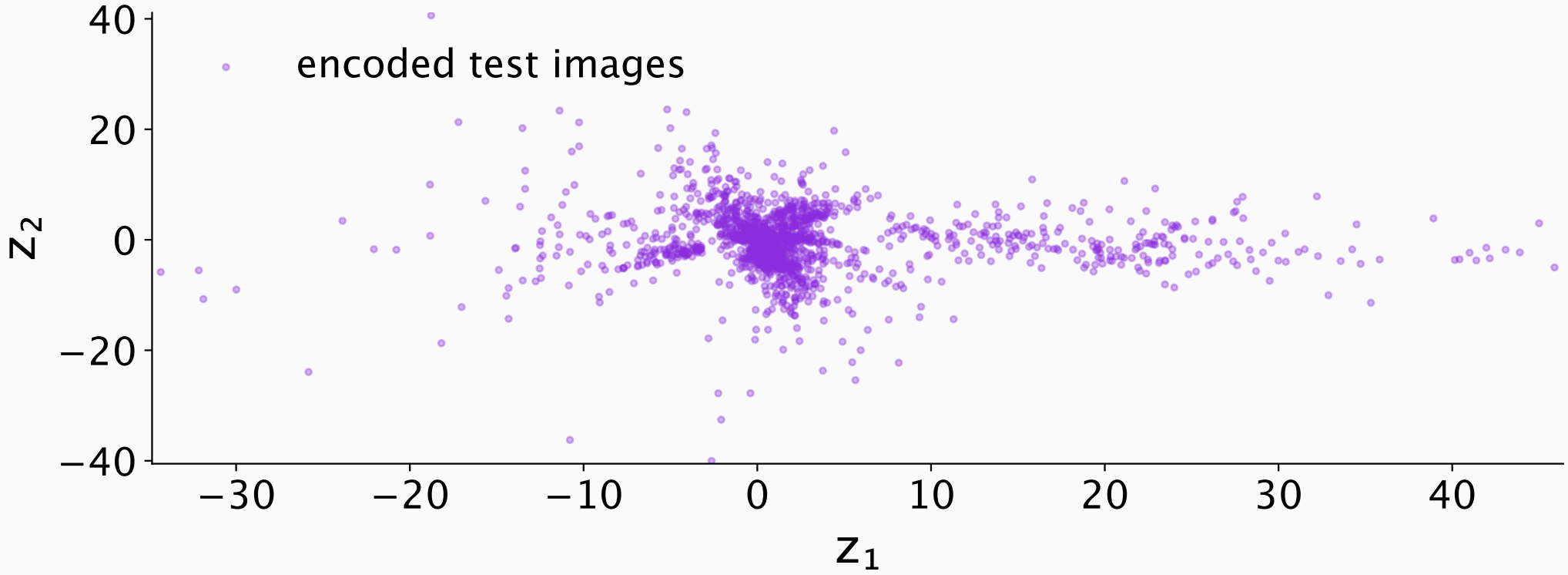


Existing AE decoder at ten fixed N(0,I) draws.

Not necessarily: these codes were not guaranteed to resemble training codes.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Where did the training objective place the codes?

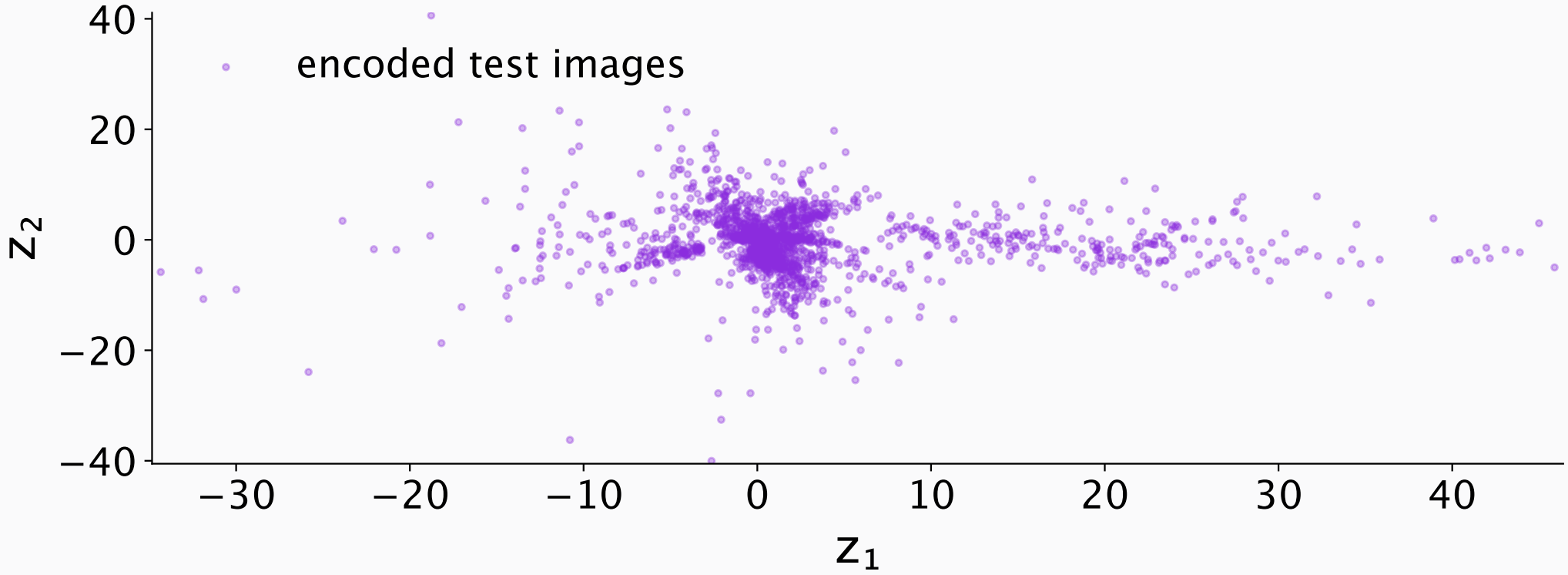


Encoder outputs for 2,000 held-out images; no latent regulariser.

> Is there any reason this cloud must be centred at zero?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Where did the training objective place the codes?



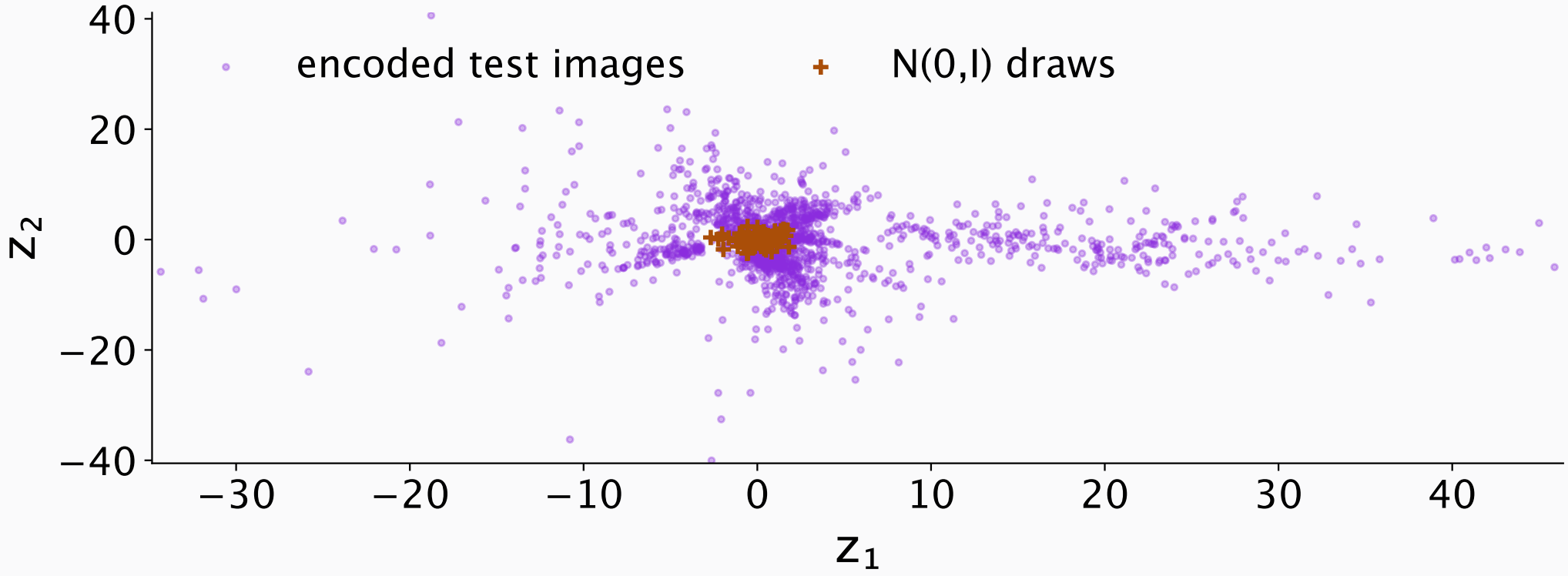
Encoder outputs for 2,000 held-out images; no latent regulariser.

The axes and units are learned. The objective never asked for a standard normal cloud.

> No. Reconstruction alone leaves its location and scale unconstrained.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Our proposed prior misses the learned code geometry

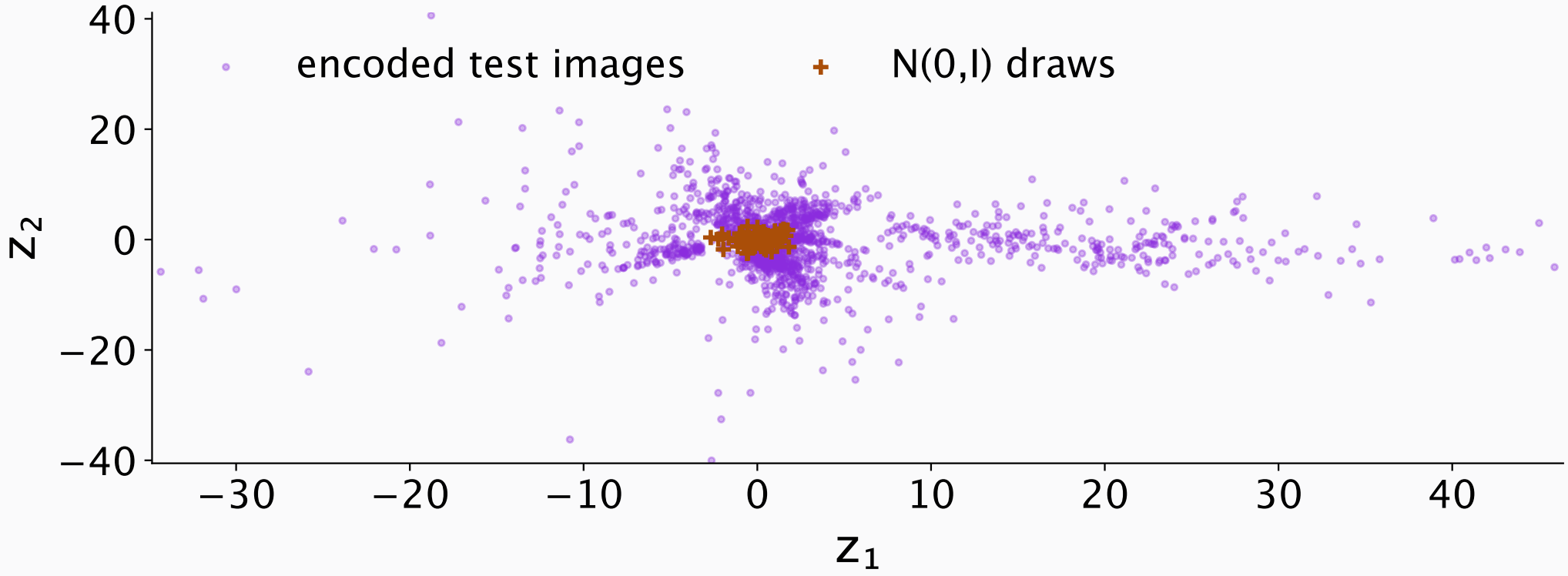


Purple: encoded test images. Amber +: fixed N(0,I) draws. Same axes.

> What changed from the previous figure?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Our proposed prior misses the learned code geometry



Purple: encoded test images. Amber +: fixed N(0,I) draws. Same axes.

Successful reconstruction did not choose a compatible sampling distribution.

> Only the prior draws were added; the encoded data and axes stayed fixed.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Two inputs to the same decoder

Code source	What the experiment tests
$E_\varphi(x)$ for a test image	Reconstruction
A fresh $N(0,I)$ draw	Compatibility of this prior and decoder

> Why can one experiment work while the other fails?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Two inputs to the same decoder

Code source	What the experiment tests
Eφ(x) for a test image	Reconstruction
A fresh N(0,I) draw	Compatibility of this prior and decoder



The second experiment, with the exact same decoder weights.

> The decoder is trained around encoded data, not necessarily every proposed random code.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Interpolation supplies a different rule for z

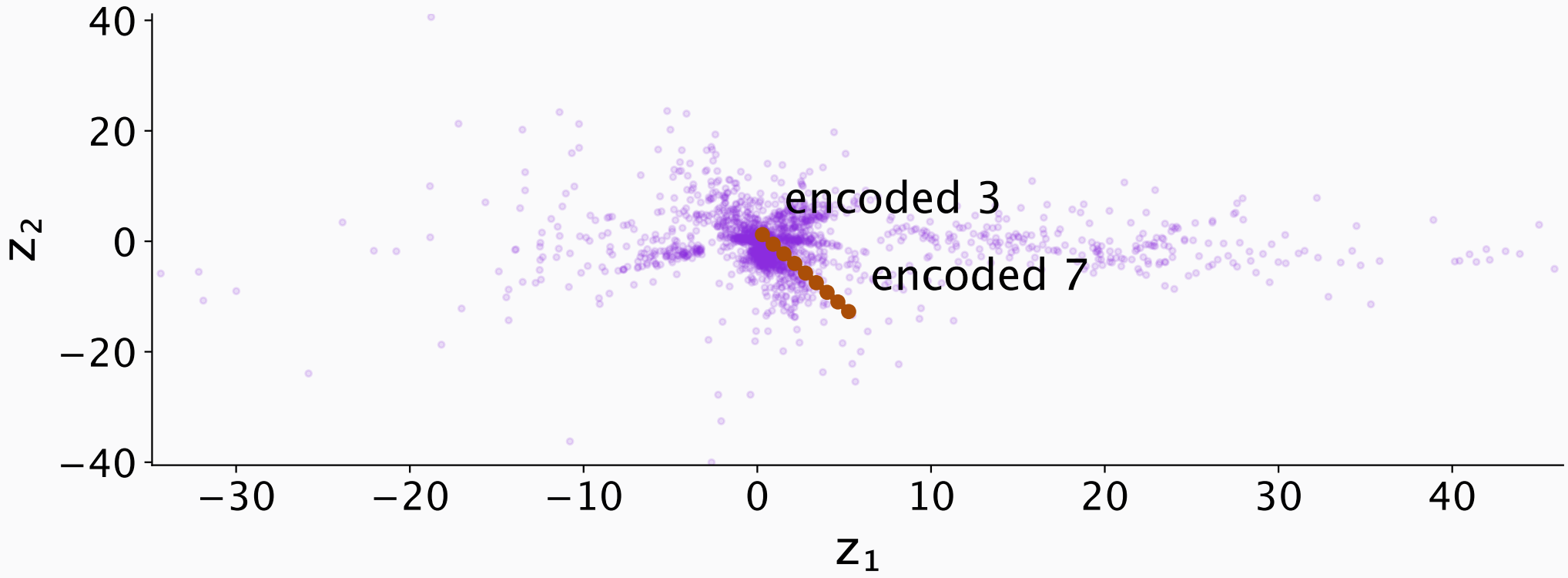
$$z(t) = (1-t)E_\phi(x_a) + tE_\phi(x_b), \quad 0 \le t \le 1$$

> Is a point on this path a standard-normal draw?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Interpolation supplies a different rule for z

$$z(t) = (1-t)E_\phi(x_a) + tE_\phi(x_b), \quad 0 \le t \le 1$$



Path from the held 3 to a held-out 7, on the same code axes.

> No. It is constructed from two encoded examples and a chosen t.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A smooth path can produce a smooth visual change



t=0 to t=1 in nine evenly spaced steps.

What conclusion does this sequence support?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A smooth path can produce a smooth visual change



t=0 to t=1 in nine evenly spaced steps.

This tests continuity along one selected path, not the quality of arbitrary prior samples.

> The decoder changes along this path; it does not establish global sampling quality.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A density fitted to codes is a legitimate extra model

$$z_i = E_\phi(x_i), \quad \text{fit } r_\gamma(z) \text{ to these codes}$$

> What additional fitting task have we introduced?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A density fitted to codes is a legitimate extra model

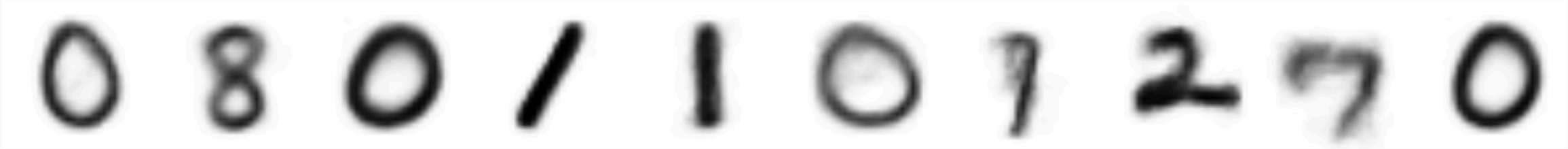
$$z_i = E_\phi(x_i), \quad \text{fit } r_\gamma(z) \text{ to these codes}$$

$$z_{\text{new}} \sim r_\gamma, \quad \hat{x} = \text{sigmoid}(G_\theta(z_{\text{new}}))$$

> Learning a distribution over encoded images.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A fitted Gaussian changes where codes are drawn

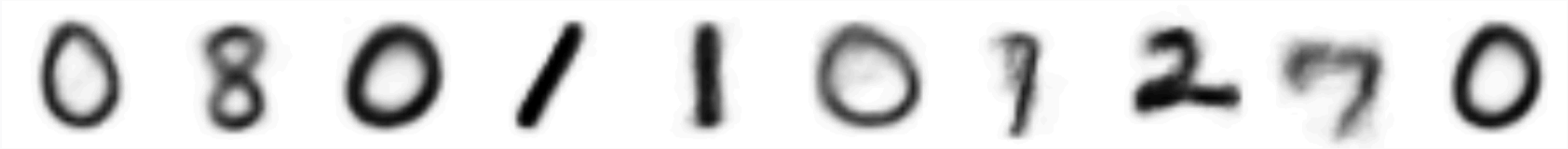


A Gaussian fitted to codes of 256 training images; ten new draws.

> Does fitting one Gaussian guarantee that it captures every code cluster?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

A fitted Gaussian changes where codes are drawn



A Gaussian fitted to codes of 256 training images; ten new draws.

The extra density may still miss holes or multiple groups. This is a separate fitted model, trained only on training codes.

> No. One Gaussian may miss multiple groups, even after its mean and covariance are fitted.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Read what the objective actually asks for

$$\min_{\phi,\theta} \; \frac{1}{N} \sum_{i} \ell(x_i, \mathrm{sigmoid}(G_\theta(E_\phi(x_i))))$$

> Which part of this expression defines a latent prior?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Read what the objective actually asks for

$$\min_{\phi,\theta} \ \frac{1}{N}\sum_{i} \ell(x_i, \mathrm{sigmoid}(G_\theta(E_\phi(x_i))))$$

There is no term specifying that encoded images follow N(0,I).

> None. The code distribution is an outcome of training, not a prescribed prior.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Which claim have we earned?

Claim	Supported by reconstruction alone?
The model reconstructs these test images	Measured here
The codes follow N(0,I)	No
Every random code gives a plausible digit	No

> What would you plot before decoding a proposed prior?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Which claim have we earned?

Claim	Supported by reconstruction alone?
The model reconstructs these test images	Measured here
The codes follow N(0,I)	No
Every random code gives a plausible digit	No

Match each claim to the experiment that could test it.

> The encoded data and prior draws on exactly the same axes.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

An AE code is a computed value

$$z = E_\phi(x)$$

> What would a distribution-valued encoder need to output?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

An AE code is a computed value

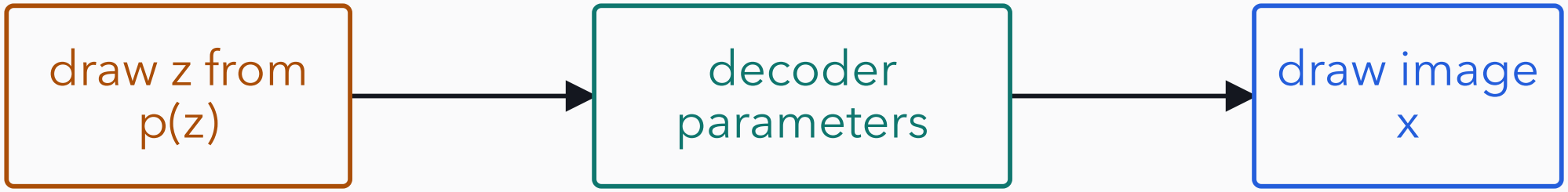
$$z = E_\phi(x)$$

This deterministic mapping is not yet an approximate posterior distribution qφ(z|x).

> Parameters describing uncertainty, such as a mean and scale.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

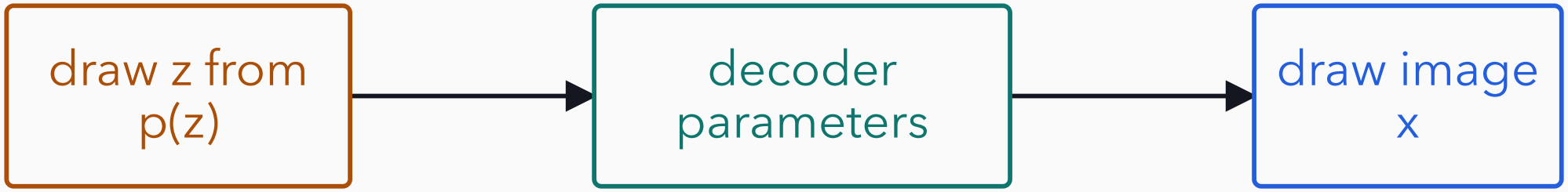
Choose a sampling story before learning inference



> Which earlier lecture prepared us for that inference problem?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Choose a sampling story before learning inference



For an observed image, we must then infer which z could have generated it.

> Lecture 1: fit a tractable approximation to a posterior.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Name the experiment from its input

Input to decoder	Your label
Code computed from an observed image	?
Line segment between two encoded images	?
Fresh draw from a stated distribution	?

> Which one was the unresolved challenge for our ordinary AE?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Name the experiment from its input

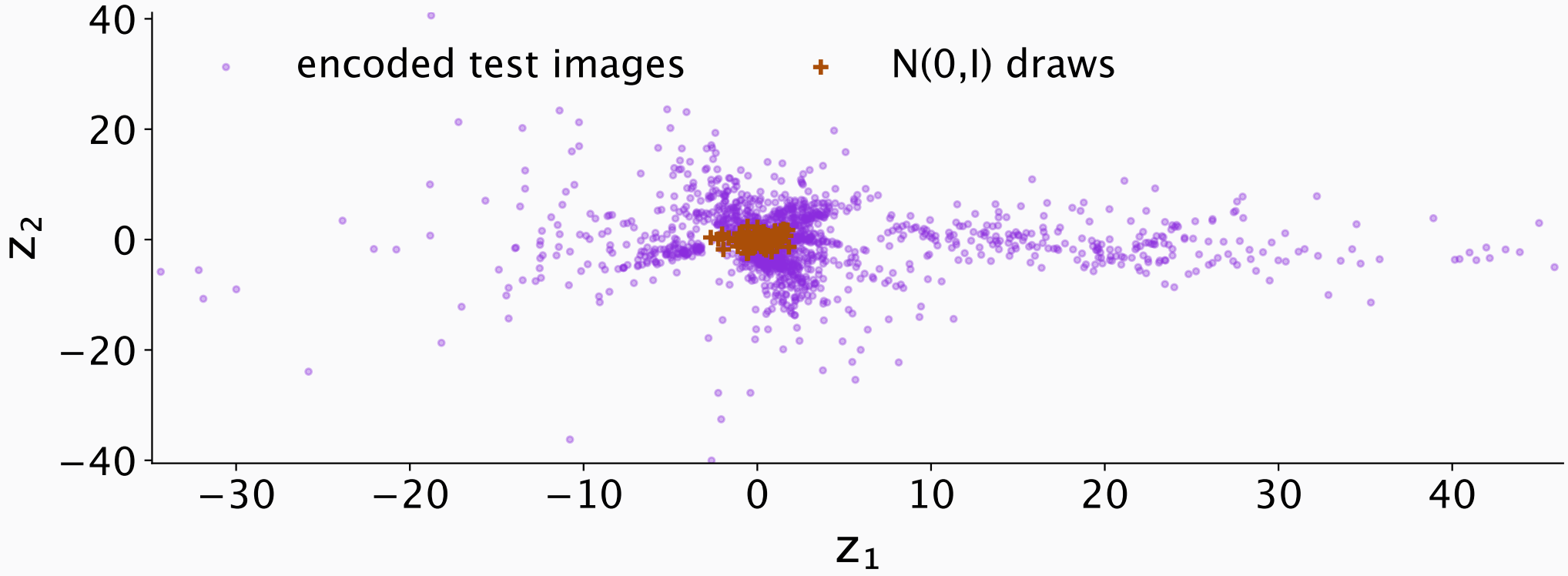
Input to decoder	Your label
Code computed from an observed image	?
Line segment between two encoded images	?
Fresh draw from a stated distribution	?

Reconstruction · interpolation · sampling. Different questions, different evidence.

> Sampling from a justified code distribution.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Next: keep the sampling question visible

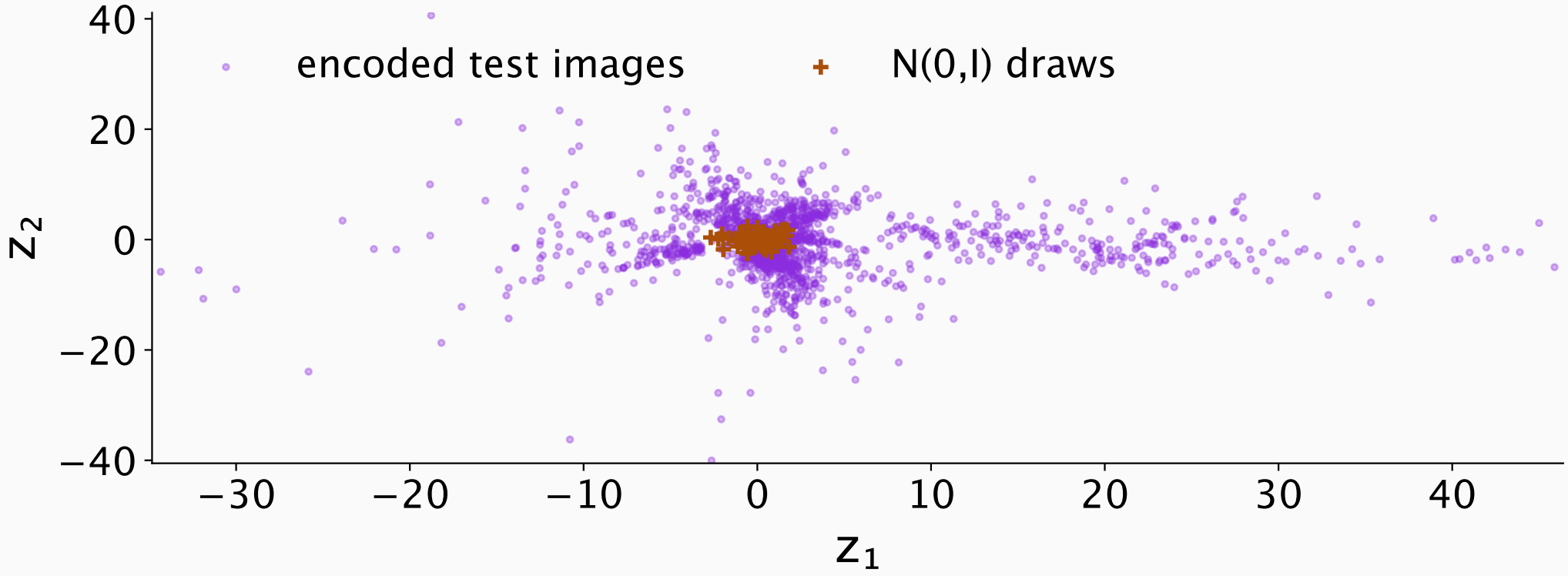


This exact frame opens the VAE lecture.

> What should our next model specify explicitly?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Next: keep the sampling question visible



This exact frame opens the VAE lecture.

Can we learn a generative model and its inference procedure together?

> A prior, a likelihood, and a way to infer latent variables from images.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Optional: why squared error appears elsewhere

$$p_\theta(x \mid z) = \mathcal{N}(x; g_\theta(z), \tau^2 I)$$

What has to be fixed for MSE to match likelihood maximisation?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Optional: why squared error appears elsewhere

$$p_\theta(x \mid z) = \mathcal{N}(x; g_\theta(z), \tau^2 I)$$

$$-\log p_\theta(x \mid z) = \frac{\|x - g_\theta(z)\|^2}{2\tau^2} + \text{constant}$$

> The Gaussian variance, with a consistent reduction.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Optional: what the mixture fitting step did

$$r_{ic} = \frac{\pi_c \mathcal{N}(x_i; \mu_c, \sigma_c^2)}{\sum_k \pi_k \mathcal{N}(x_i; \mu_k, \sigma_k^2)}$$

> Is c observed in the fitting data?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Optional: what the mixture fitting step did

$$r_{ic} = \frac{\pi_c \mathcal{N}(x_i; \mu_c, \sigma_c^2)}{\sum_k \pi_k \mathcal{N}(x_i; \mu_k, \sigma_k^2)}$$

Alternate soft component assignments and weighted parameter estimates. Our fixture uses 40 such updates.

> No; responsibilities are model-based soft assignments.

Optional: codes are not uniquely identified

$$z' = Az, \qquad G'_\theta(z') = G_\theta(A^{-1}z')$$

> Why does reconstruction alone leave code scale ambiguous?

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.

Optional: codes are not uniquely identified

$$z' = Az, \qquad G'_\theta(z') = G_\theta(A^{-1}z')$$

For invertible A, the reconstruction can stay unchanged while the code cloud changes.

> The decoder can undo a corresponding invertible change in the encoder code.

Original teaching construction; measured figures replay local binary-MNIST checkpoints. See data/provenance.json.